

Semantic Visual Validation for OTT User Interfaces Using Deep Learning

Bhavan Kumar B R¹, Partha Sarathi Samal^{2*}, Suresh Kumar Palus³

¹ Independent Researcher, Colorado, USA. bavankrishna1@gmail.com

² Independent Researcher, Connecticut, USA. samalpartha@gmail.com

³ Independent Researcher, Pennsylvania, USA. sure.1985@gmail.com

* Corresponding author: Partha Sarathi Samal (samalpartha@gmail.com)

Abstract. Visual validation of over-the-top interfaces on connected televisions must distinguish harmless rendering variation from defects in focus, layout and transitions. This paper specifies a semantic validation framework combining focus detection, element classification, temporal analysis and design-derived test oracles in a deterministic decision rule. A synthetic benchmark emulates four platform focus conventions across four screen types and defines seven benign variation classes and six defect classes. A non-learned implementation of the decision rule is compared with pixel-diff and structural similarity on 624 held-out instances. The supplied evaluation reports a false positive rate of 6.8%, defect recall of 95.5%, a harmonic mean of precision and recall of 0.939 and mean latency of 96.8 milliseconds per frame on one processor core. Pixel-diff reports 70.5% false positives and 69.8% recall, while structural similarity reports 65.8% and 62.8%. These measurements concern the synthetic renderer and rule-based implementation. They do not establish performance of the learned components or generalization to real devices. The study describes the experimental design, attributable failure cases and requirements for subsequent real-device evaluation.

Keywords: Semantic visual validation, OTT user interfaces, Connected TV, deep learning, focus state detection, motion validation, HDR UI testing, visual test automation.

1 Introduction

Over-the-top (OTT) streaming applications on Connected TV (CTV) platforms are operated with a remote control from across a room. Nothing is clicked. The viewer moves a focus indicator from tile to tile, and the interface has to make it obvious at every moment which element is active, where the next press will lead, and what the transition between two screens means. Designers lean on scale changes, glow, animated carousels and depth cues to carry this information, and they do it on panels that render high dynamic range (HDR) content with their own brightness curves. The same application is then shipped to Roku, Android TV, Tizen and WebOS, each with a

different rendering pipeline and user interface (UI) framework, so the same design produces slightly different pixels on every device.

Visual testing practice has not caught up with this reality. The dominant approach compares a captured screenshot with an approved reference and fails the test when the number of changed pixels crosses a threshold. On a deterministic desktop form this is adequate. On a CTV home screen it is noisy in both directions. Anti-aliased text, a carousel captured a few frames later than the reference, a different tone mapping curve, or a resolution scale step produce large pixel deltas while the interface is functionally correct. Recent benchmarking work on web UI regression describes pixel comparison as semantically blind for exactly this reason, and reports that trained vision models already suppress non-meaningful changes more selectively than pixel diffs do [9]. At the same time the defects that a viewer actually notices, a focus ring drawn on the wrong tile, a primary button that lost contrast against a bright hero image, a menu that overlaps the content rail, may move very few pixels and pass a pixel threshold without complaint.

Deep learning models for screen understanding have matured quickly. Vision-only parsers such as OmniParser detect interactable elements and describe their function from a screenshot alone [6], and native GUI agent models such as UI-TARS and the GUI-Owl family perceive screenshots end to end and act on them across desktop, mobile and browser targets [7, 14]. Those systems are built to operate an interface, not to judge one. They rarely model focus as a first-class state, they work on single frames rather than motion sequences, and they have been evaluated on desktop and mobile layouts rather than on ten-foot television interfaces. Software engineering research has started to apply multimodal models to GUI testing, for example by comparing pre-change and post-change screens in the light of a natural language change intent [10] or by letting agents explore an application for defects [11], but neither line of work targets focus semantics or animated transitions on CTV.

This paper proposes a Semantic Visual Validation framework that closes that gap. Frames are captured from real devices, normalized so that platform-specific rendering does not bias the models, and passed to three cooperating analyzers: a context-aware focus detector, a semantic UI element classifier built from convolutional neural networks (CNNs) and vision transformers (ViTs), and a temporal analyzer that reasons over motion sequences. Their outputs are scored against an intent-derived oracle and reduced to a deterministic pass or fail. Fig. 1 shows the overall arrangement. The main contributions of this work are as follows.

- A formal decision model for semantic visual validation of OTT interfaces, in which focus correctness, element-level semantic match and temporal smoothness are each defined as scored quantities and combined into a single deterministic verdict.
- A context-aware focus state detector that fuses luminance contrast, spatial prominence and a learned focus probability, and therefore tolerates platform-specific focus styles without per-platform rules.
- A temporal validation component that checks the smoothness of focus trajectories and the continuity of frame-to-frame change during carousel scrolling and screen transitions, which static screenshot comparison cannot do.

- Automatic generation of visual test oracles from design artifacts, so that a layout change updates the expected semantics rather than a stored bitmap.
- An evaluation protocol with a taxonomy of benign variations and injected defects, a reproducible synthetic Connected TV benchmark that emulates four platform focus conventions, and a comparison of a non-learned instantiation of the decision rule against pixel-diff and structural similarity on false positive rate, recall, precision, F1 and latency.

The rest of the paper is organized as follows. Section 2 reviews screenshot-based UI testing, the specific difficulties of ten-foot interfaces, and recent deep learning work on GUI understanding. Section 3 presents the architecture and the mathematical definitions behind each stage. Section 4 describes the deep learning models and the generated test oracles. Section 5 defines the experimental setup and the benchmark, Section 6 reports and discusses the results, and Section 7 concludes with limitations and future work.

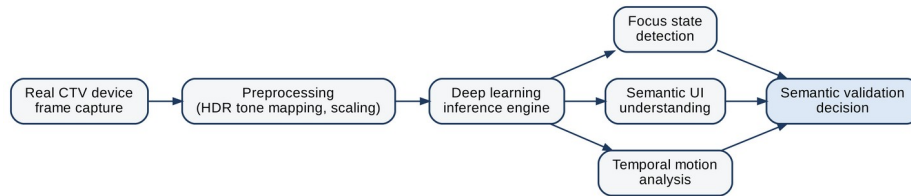


Fig. 1. Overall architecture of the Semantic Visual Validation framework.

2 Background and Related Work

This section reviews the three bodies of work the framework draws on: screenshot-based visual regression testing, the properties of ten-foot interfaces that make such testing fragile, and deep learning for GUI understanding.

2.1 Screenshot and pixel-diff based UI testing

Visual regression testing captures a screenshot of a screen, compares it with an approved baseline and routes any difference to a reviewer. Open-source and continuous integration (CI) pipelines mostly use per-pixel comparison with a tolerance, sometimes with masked regions for dynamic content. Fig. 2 shows the flow and where false failures enter it. The method presumes that visual correctness is equivalent to pixel similarity. On a CTV screen the presumption fails: font hinting differs by platform, a carousel is captured mid-scroll, an HDR panel remaps luminance, and every one of those changes many pixels while changing nothing a viewer would call a bug. Commercial tools such as AppliTools Eyes and Percy have added learned comparison to reduce this noise, but they publish no evaluation, and a 2026 benchmark study found that the problem of separating meaningful regressions from rendering noise remains largely unexplored in the open literature [9]. Pixel methods also have a blind spot on

the other side: they cannot say whether focus sits on the intended element or whether the visual hierarchy still leads the eye to the primary action, because those properties are not functions of pixel counts.

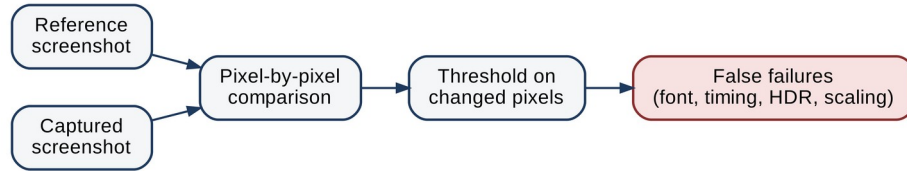


Fig. 2. Pixel-diff based UI testing and the sources of false failures.

2.2 Visual validation challenges in ten-foot UI experiences

Television interfaces are designed for a viewer sitting about ten feet from the screen, which changes what matters. Fig. 3 summarizes the properties that drive validation difficulty. Navigation is focus-driven, so the visibility of the active element is a correctness property rather than a styling detail. Carousels, tile scaling and screen transitions are animated, and the meaning of a transition is spread over a sequence of frames rather than contained in any one of them. HDR output introduces brightness and color variance between panels that a reference bitmap captured on one device cannot anticipate. Finally, each platform has its own focus style and rendering pipeline, so the same design token produces different pixels on Roku and on Tizen. Earlier work on spotting UI display issues from screenshots, such as Owl Eyes [3], addressed text overlap and missing images on mobile, but did not model focus or motion because mobile interfaces do not depend on them.

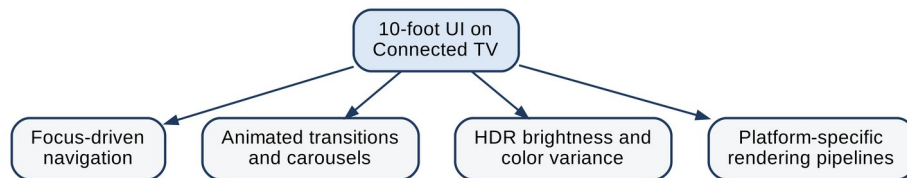


Fig. 3. Visual complexity and validation challenges in ten-foot UI experiences.

2.3 Deep learning in UI understanding and computer vision

Deep learning has made object detection, semantic segmentation and scene classification reliable enough to run in production pipelines, and vision transformers [5] extended that reliability to global layout reasoning. Large screen datasets such as Rico [4] made it possible to train models that recognize UI components and infer structure from pixels alone. OmniParser [6] fine-tuned an interactable-region detector and an icon captioning model to turn a screenshot into a structured element list and showed that this parse alone lifts a general vision-language model to state-of-the-art

grounding. The most recent generation goes further: UI-TARS [7] is a native GUI agent that perceives screenshots end to end, and GUI-Owl-1.5 reports state-of-the-art grounding on ScreenSpot-Pro across desktop, mobile and browser targets [14]. A 2025 survey of GUI agents built on foundation models catalogs this progress and notes that evaluation has centered on task completion rather than on judging interface correctness [8]. Fig. 4 shows how these models fit into a validation pipeline: frames go in, and semantic structure comes out.



Fig. 4. Deep learning based semantic understanding of OTT user interfaces.

2.4 Gaps in semantic and context-aware visual validation

The pieces exist, but not the assembly. Screen parsers and GUI agents detect and ground elements; they do not decide whether a screen is correct with respect to an intent. Software engineering work that does make correctness judgments, such as RippleGUItester [10] and GUITester [11], targets desktop and mobile applications, reasons from single screenshots, and does not model focus as a state. No published system combines focus awareness, motion analysis over frame sequences and intent-based validation in one deterministic pipeline, and none has been evaluated on ten-foot HDR interfaces. That is the gap this paper addresses.

3 Proposed Semantic Visual Validation Architecture

The framework is a layered pipeline that separates perception from judgment. Perception turns device frames into semantic structure. Judgment compares that structure with an intent-derived oracle and produces a verdict. This section defines both layers and the quantities that connect them.

3.1 Architectural overview

Fig. 5 shows the pipeline. A real CTV device is driven through a test scenario. A capture module reads frames from the device frame buffer or from an HDMI capture card. Preprocessing normalizes resolution, tone and noise. A deep learning inference engine produces three kinds of evidence: the focus state, the set of semantic elements with their positions, and the motion characteristics of the sequence. The validation engine scores each against the oracle for the current intent and combines the scores into a deterministic decision. Because the validation logic consumes semantic structure rather than pixels, controlled visual variability in the input does not reach the verdict, while semantic defects do.

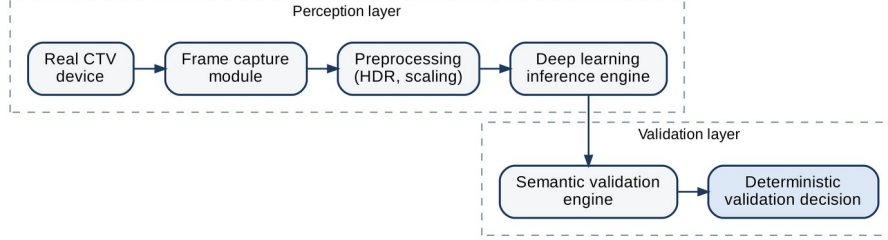


Fig. 5. Layered Semantic Visual Validation pipeline. Rectangles denote processing stages; the dashed groups mark the perception and validation layers.

3.2 Frame capture and preprocessing

Frames are acquired from the device rather than from an emulator, since platform rendering is part of what is under test. Fig. 6 shows the preprocessing chain. Each frame is first brought to a common resolution and aspect ratio. HDR frames are then tone mapped so that the models see a consistent luminance range regardless of the panel. The global photographic operator of Reinhard et al. [13] is used, which maps scene luminance L to display luminance according to

$$L_d = \frac{L}{1 + L} \quad (1)$$

after which the frame is normalized per channel and resized to the model input size $W \times H$:

$$\hat{I} = \text{resize} \left(\frac{I_{tm} - \mu}{\sigma}, W \times H \right) \quad (2)$$

where I_{tm} is the tone-mapped frame and μ and σ are the channel mean and standard deviation estimated on the training set. A light denoising step removes capture artifacts. The chain deliberately preserves edges and local contrast, because focus rings and motion boundaries are exactly the features later stages depend on [13].



Fig. 6. Frame capture and preprocessing pipeline: resolution normalization, HDR tone mapping and artifact reduction ahead of inference.

3.3 Semantic intent-to-visual mapping engine

The mapping engine is the core of the judgment layer. A user intent I , such as play, select or navigate, is associated with an expected element set

$$E(I) = \{(c_k, \rho_k) : k = 1, \dots, K\} \quad (3)$$

where c_k is a semantic class (for example primary button, focused tile, rail tile) and ρ_k is its expected normalized position on screen. The oracle $E(I)$ is generated from design artifacts as described in Section 4.4. The perception layer supplies a set of detected elements with predicted classes $\hat{c}_i$ and positions $\hat{\rho}_i$. The semantic match score is the fraction of expected elements that have a detected element of the same class within a position tolerance δ :

$$M(I) = \frac{1}{|E(I)|} \sum_{k=1}^K \max_i \mathbb{1}[\hat{c}_i = c_k] \cdot \mathbb{1}[\|\hat{\rho}_i - \rho_k\| \leq \delta] \quad (4)$$

Position enters only through a tolerance, so a tile that moved by a few pixels because a platform rounds differently still matches, while a missing primary button or a button demoted to a secondary style does not. Fig. 7 shows how the intent, the mapping engine, static validation and temporal validation feed the final decision.

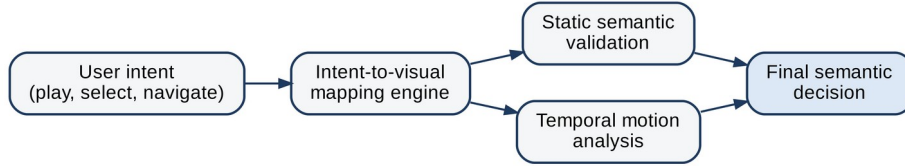


Fig. 7. Semantic intent-to-visual mapping and temporal validation flow.

3.4 Temporal analysis for motion and transition validation

Static comparison cannot judge a carousel scroll or a screen transition, because their correctness lives in how frames relate to one another. The temporal analyzer operates on a sequence of T normalized frames $I_1, \dots, I_T$. Let x_t denote the position of the focus indicator in frame t , obtained from the focus detector of Section 4.1 or, mid-transition, from the motion tracker of Section 4.3. Smoothness is the worst second difference of the trajectory relative to the distance it travels:

$$S = 1 - \frac{\max_t \|x_{t+1} - 2x_t + x_{t-1}\|}{\|x_T - x_1\|} \quad (5)$$

An eased motion that accelerates and settles has a small worst-case second difference compared with the span it covers and scores close to one; a focus ring that jumps, disappears for a frame, or lands on the wrong tile produces a second difference comparable to the whole span and pulls S to zero or below. Frame-to-frame change is measured as the mean absolute intensity difference

$$\Delta_t = \frac{1}{WH} \|I_{t+1} - I_t\|_1 \quad (6)$$

and continuity is declared when no single step is far larger than the typical step, $G = \mathbb{1}[\max_t \Delta_t \leq \kappa \cdot \text{median}_t \Delta_t]$. A dropped frame or a flash to black during a transition violates this condition. The temporal analyzer therefore checks both where focus goes and how the screen gets there.

4 Deep Learning Models and Core Components

Four components produce the evidence that the decision rule consumes. Fig. 8 shows how they connect. This section defines each component and the final decision rule.

4.1 Context-aware focus state detection

Focus visibility is the single most important usability property of a television interface, and it is also the one that varies most between platforms. Rule-based detectors that look for a white border break as soon as a platform uses scale or glow instead. The proposed detector scores every candidate element i with three terms: its luminance contrast against its neighbors, its spatial prominence, and a learned focus probability:

$$s_i = \alpha C_i + \beta P_i + \gamma g_\phi(r_i), \quad \text{with } C_i = \frac{L_i - \bar{L}_{N(i)}}{L_i + \bar{L}_{N(i)}}, P_i = \frac{A_i}{\bar{A}_{N(i)}} \quad (7)$$

Here L_i and A_i are the mean luminance and rendered area of element i , the barred quantities are the same statistics over its spatial neighborhood $N(i)$, r_i is the image crop of the element, and g_ϕ is a CNN trained to output the probability that a crop is in the focused state. The weights α , β and γ are fitted on labeled frames. The predicted focus index is $\hat{f} = \text{argmax}_i s_i$ and the focus check for intent I is $F = \mathbb{1}[\hat{f} = f^*(I)]$, where $f^*(I)$ is the element that the oracle says should hold focus. Because contrast and prominence are relative to the neighborhood, the detector adapts to a platform that renders focus as scale, glow or border without a per-platform rule.

4.2 Semantic UI element classification using CNNs and vision transformers

Element classification uses a two-stage design similar in spirit to vision-only screen parsers [6]. A CNN detector proposes element regions r_i ; a ViT classifier then assigns each region a semantic class by

$$p(c | r_i) = \text{softmax}(f_\theta(r_i)), \hat{c}_i = \text{arg max}_c p(c | r_i) \quad (8)$$

The class vocabulary covers the elements that carry meaning on a CTV screen: tile, focused tile, primary button, secondary button, rail title, navigation menu item, dialog, and playback control. The CNN captures local texture such as a focus border, while the ViT relates a region to the whole layout, which is what allows a button to be recognized as primary from its position and weight rather than from its exact color. Both models

are trained on frames from several platforms so that a shared visual grammar, rather than one platform style, is learned [5].

4.3 Motion and gradient consistency validation

The motion analyzer implements Eqs. (5) and (6) on frame sequences and adds a gradient consistency check for animated gradients and HDR playback surfaces. Dense optical flow from a RAFT model [12] provides the per-pixel motion field used to track the indicator between frames when the focus detector alone is ambiguous, for example while the indicator sits between two tiles during a fling; a cheaper stand-in, used in Section 6, is the centroid of inter-frame change, which is adequate when the indicator is the only moving element. Color consistency during an animation is checked by requiring that the mean hue of a tracked region drifts by no more than a fixed tolerance over the sequence. Together these checks confirm that transitions are smooth, that focus is continuous, and that brand gradients do not band or flicker on HDR panels.

4.4 Generated visual test oracles from design artifacts

Maintaining bitmap baselines is the hidden cost of visual regression testing. Every intentional redesign invalidates them. The framework replaces bitmaps with semantic oracles generated from design files such as Figma or Sketch exports. A generative model reads the design specification for a screen, identifies the elements and their roles, and emits the expected element set $E(I)$ of Eq. (3) together with the expected focus element $f^*(I)$ for each intent. A layout change then updates the oracle automatically; only a change in interaction semantics requires a human review. The oracle is the ground truth for Eqs. (4) and (7).

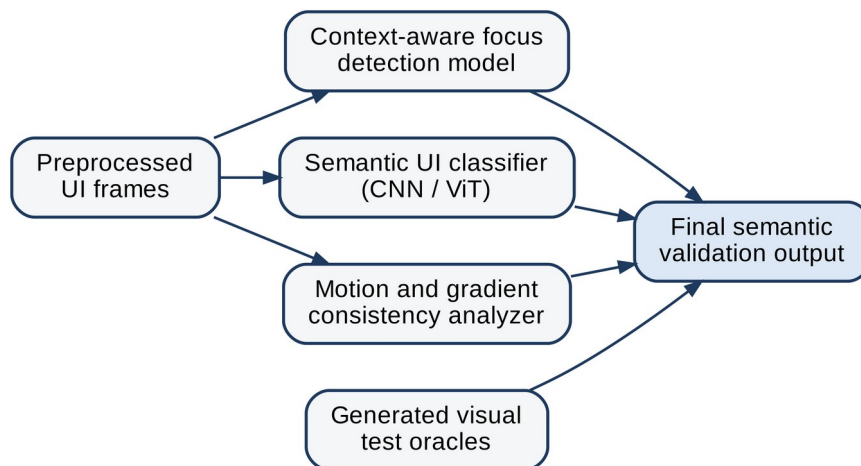


Fig. 8. Deep learning models and core components feeding the final decision.

4.5 Deterministic decision rule

The four analyzers reduce to a single verdict. A frame or sequence passes when focus is correct, the semantic match reaches its threshold, the trajectory is smooth enough and the sequence is continuous:

$$V = F \cdot \mathbb{1}[M(I) \geq \tau_M] \cdot \mathbb{1}[S \geq \tau_S] \cdot G \quad (9)$$

Every term is a function of scored quantities and fixed thresholds, so the same frames always yield the same verdict, and a failure can be attributed to the term that caused it. That attribution is what a tester sees in the report: a focus failure on element k , a missing primary button, or a discontinuity at frame t .

5 Experimental Setup

This section defines the benchmark, the defect taxonomy, the methods compared and the metrics. The protocol measures false positives on harmless variation and misses on real defects separately, since a method can look good on one while failing the other.

5.1 Synthetic Connected TV benchmark and defect taxonomy

Real-device captures were not available for this study, so the evaluation uses a synthetic benchmark whose renderer emulates the properties that make CTV validation hard. Four platform styles differ in focus convention (a white border, an eight percent scale-up with a thin outline, a blurred glow halo, and a colored border combined with a five percent scale-up), in font family, in panel gamma and in tile palette. Four screen types (home, browse, detail and playback) are laid out with hero gradients, content rails of six tiles, rail titles, primary and secondary buttons and a playback control bar. Frames are rendered at 960 by 540 pixels, half of 1080p. For each of the sixteen platform and screen pairs, four base screens are generated with different focus targets and rail titles; one is assigned to a validation split used for threshold tuning and three to a held-out test split. Every base screen produces one reference frame and a semantic oracle $E(I)$, then thirteen test instances: seven benign variations that should pass and six injected defects that should fail. Table 1 lists the classes. Transition classes B7, D5 and D6 are sequences of sixteen frames in which focus moves from one element to the next with an eased trajectory; the reference for those classes is the same sequence rendered without the defect. The benchmark totals 832 instances, 624 of them in the test split, and the generator and oracles are released with the paper so the numbers can be reproduced.

Table 1. Benign variation classes (expected verdict PASS) and injected defect classes (expected verdict FAIL).

Code	Description	Typical cause on CTV
B1	Global brightness gain of ± 10 to ± 15 percent	Panel tone mapping, HDR to SDR fallback
B2	Whole-layout translation of up to 3 px	Platform rounding, safe-area offsets
B3	Gaussian blur of 0.7 px emulating different text rasterizers	Font anti-aliasing and hinting per platform
B4	Resolution scale to 640 by 360 and back	Device output mode, capture card
B5	Carousel captured 8 to 21 px into its settling animation	Animation timing, capture latency
B6	Different artwork inside every tile (stripes, disc or poster)	Personalized or scheduled content
B7	Correct focus transition rendered at 0.9 brightness gain	Panel brightness during navigation
D1	Focus indicator missing	Focus style regression
D2	Focus on a different element than the intent requires	Navigation logic regression
D3	Primary button missing or demoted to secondary style; a rail tile or control missing on screens without a primary button	Hierarchy regression, theme error
D4	Tile drawn over its neighbor, rail title drawn over the tiles, or a label running past its button edge	Layout regression, localization overflow
D5	Focus jumps to a distant element for four frames, or the indicator drops out for two frames	Animation regression
D6	Hero gradient quantized to eight bands, or a hue flicker every fifth frame	HDR rendering regression

5.2 Methods compared

Table 2 lists the methods evaluated. Pixel-diff is the CI default and fails a frame when the fraction of pixels whose intensity differs from the reference by more than ϵ exceeds τ_p :

$$D_{pix} = \frac{1}{WH} \sum_{x,y} \mathbb{1}[|I(x,y) - R(x,y)| > \epsilon], \quad \text{fail if } D_{pix} > \tau_p \quad (10)$$

Structural similarity (SSIM) [1] represents perceptual comparison and fails a frame when the index drops below a threshold. A sequence fails under either baseline if any of its frames fails against the corresponding reference frame. Both thresholds were set to the value that maximized F1 on the validation split ($\epsilon = 25$ of 255, $\tau_p = 0.002$, SSIM threshold 0.99), which is the best setting among the thresholds tested on the validation split.

The proposed method is evaluated as a non-learned instantiation of Sections 3 and 4, because the learned focus term g_ϕ and the CNN and ViT classifiers require training data that the synthetic benchmark cannot substitute for without circularity. In this instantiation $\gamma = 0$ in Eq. (7); each oracle element is verified by rule within the position tolerance δ (foreground coverage and edge presence on all four sides for tiles, bright saturated coverage for the primary button, local brightness contrast for playback controls, text pixel count for rail titles, and a check that no label runs across its button edge); the ring statistic in Eq. (7) is the second-lowest of the four side means so that a rail title above a tile does not count as an indicator; and the transition tracker of Section 4.3 uses the centroid of inter-frame change. Everything else, including Eqs. (4), (5), (6) and (9), is implemented as written. Learned perceptual distance, the OmniParser element-match baseline and the vision-language judge described in Section 2 were not run: the perceptual model needs pretrained weights that were not available in the evaluation environment, and the parser and agent models are trained on real screenshots, so their scores on synthetic renders would measure domain shift rather than validation ability. They remain the planned comparison for the real-device study.

Table 2. Methods evaluated on the benchmark.

Method	Category	What it compares
Pixel-diff	Pixel baseline	Changed-pixel fraction against the reference, Eq. (10); threshold tuned on validation
SSIM [1]	Perceptual baseline	Structural similarity against the reference; threshold tuned on validation
Proposed (rule-based instantiation)	Semantic validation	Focus, semantic match and temporal terms of Eq. (9) against the oracle; no learned component

5.3 Metrics and threshold tuning

A verdict of FAIL on a benign instance is a false positive; a verdict of PASS on a defective instance is a false negative. With TP, FP, TN and FN counted over the test split,

$$\begin{aligned} FPR &= \frac{FP}{FP + TN}, & Recall &= \frac{TP}{TP + FN} \\ Precision &= \frac{TP}{TP + FP}, & F1 &= \frac{2 \cdot Precision \cdot Recall}{Precision + Recall} \end{aligned} \quad (11)$$

False positive rate is reported on classes B1 to B7, recall on classes D1 to D6, and precision and F1 over the whole split. Latency is wall-clock time per frame on a single CPU core with no GPU, averaged over all frames including those inside sequences. The parameters of the proposed rule (θ_i , α , β , τ_M , δ , κ , τ_S and the gradient step limit) were tuned on the validation split only, by a keep-or-discard loop that mutates one parameter at a time and keeps a mutation when it does not reduce the number of five binary criteria met (validation FPR at most 5 percent, recall at least 90 percent, every benign class at most 10 percent false alarms, every defect class at least 75 percent detection, latency under 100 ms) and improves F1; the loop ran for 16 iterations and its log is released with the code. The test split was never used for tuning.

6 Results and Discussion

This section reports the outcome of the protocol on the held-out test split and discusses what it does and does not show. The test split contains 624 instances: 336 benign and 288 defective, of which 144 are sixteen-frame sequences.

6.1 Overall comparison

Table 3 compares the three methods on the four quality metrics and on latency. Fig. 9 shows the same data per class.

Table 3. Results on the test split. FPR is computed on benign classes B1 to B7, recall on defect classes D1 to D6, precision and F1 over the whole split. Latency is per frame on one CPU core.

Method	FPR (%)	Recall (%)	Precision (%)	F1	Latency (ms)
Pixel-diff	70.5	69.8	45.9	0.554	10.5
SSIM [1]	65.8	62.8	45.0	0.525	36.3
Proposed (rule-based)	6.8	95.5	92.3	0.939	96.8

The two baselines fail in the way Section 2.1 predicts. The threshold that maximizes pixel-diff F1 on the validation split is very tight (0.2 percent of pixels), and at that

setting it flags 70.5% of the benign instances while still missing 30.2% of the defects; the looser one percent threshold that CI pipelines typically use passes more of the benign variation but, on the validation split, detected only a third of the defects. No threshold separates the two populations, because the number of changed pixels does not track whether the interface is correct: a three pixel layout shift changes every edge on the screen, a missing focus ring changes a thin band. SSIM at its best threshold behaves the same way, flagging 65.8% of benign instances for 62.8% recall. Both end near an F1 of 0.55, which is roughly what flagging almost everything earns on a split with 288 defective out of 624 instances. The proposed rule reaches an FPR of 6.8% and a recall of 95.5%, an F1 of 0.939, because it never compares pixels with a reference bitmap: it asks whether the expected elements are present, whether focus sits where the intent requires, and whether the transition was smooth and continuous. Its latency of 96.8 ms per frame on one CPU core comes almost entirely from the exhaustive offset search in element verification and is within the budget of a device test that samples a few frames per second.

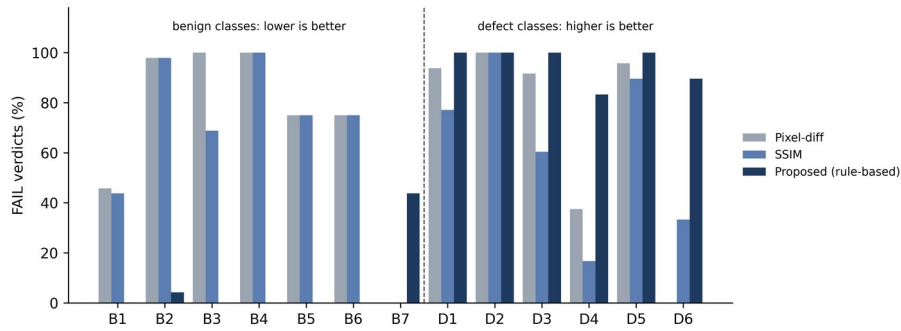


Fig. 9. Per-class false alarm rate (B1 to B7, lower is better) and detection rate (D1 to D6, higher is better) on the test split.

6.2 Per-class behavior

Table 4 gives the per-class rates behind Fig. 9. For benign classes the entry is the false alarm rate and lower is better; for defect classes it is the detection rate and higher is better.

Table 4. Per-class false alarm rate (B1 to B7) and detection rate (D1 to D6) on the test split, in percent.

Class	Pixel-diff	SSIM	Proposed (rule-based)
B1	45.8	43.8	0.0
B2	97.9	97.9	4.2
B3	100.0	68.8	0.0

Class	Pixel-diff	SSIM	Proposed (rule-based)
B4	100.0	100.0	0.0
B5	75.0	75.0	0.0
B6	75.0	75.0	0.0
B7	0.0	0.0	43.8
D1	93.8	77.1	100.0
D2	100.0	100.0	100.0
D3	91.7	60.4	100.0
D4	37.5	16.7	83.3
D5	95.8	89.6	100.0
D6	0.0	33.3	89.6

Four observations follow. First, the benign classes are where the baselines lose. Translation (B2), rescaling (B4), blur (B3) and a late carousel capture (B5) are flagged by pixel-diff in 97.9, 100.0, 100.0 and 75.0 percent of cases, and by the semantic rule in 4.2, 0.0, 0.0 and 0.0 percent, because the position tolerance δ and the tone normalization of Eq. (2) absorb exactly these variations. Changed artwork inside tiles (B6) is a false alarm for pixel-diff three times in four and never for the semantic rule, since the oracle describes a tile, not its poster. Second, on single-frame defects the tight pixel threshold does catch most missing and misplaced focus (D1, D2) and missing hierarchy elements (D3), but only because it catches nearly everything; the semantic rule reaches 100.0, 100.0 and 100.0 percent on the same classes while leaving benign frames alone, and it is the only method that detects most overlap and truncation defects (D4: 83.3 percent against 37.5 for pixel-diff and 16.7 for SSIM). Third, the transition classes split. Focus jumps and dropped frames (D5) are caught by every method, gradient banding and hue flicker (D6) by the semantic rule in 89.6 percent of cases against 0.0 for pixel-diff, whose grayscale comparison barely registers a hue change. The one class the semantic rule handles worse than the baselines is the benign transition B7, with 43.8 percent false alarms: on browse screens, which have no hero gradient, the instantiation applied the gradient step check to a column that crosses the tile rail, and on the scale-style platform the change-centroid tracker, which stands in for optical flow, produces a trajectory whose smoothness falls just under $\tau_{\{S\}}$. Both are limits of the rule-based instantiation rather than of Eqs. (5) and (6), and the deterministic design makes them visible as such: every B7 false alarm in the log is attributed to the gradient term or to S, never to focus or semantic match. Fourth, the remaining misses on D4 and D6 have identifiable causes. A label that runs past its button edge but is hidden under a focus border or a neighboring button is invisible to the edge-crossing test, and a hue flicker on the platform whose focus indicator is a large colored border is masked by the median normalization of Eq. (6), because the moving indicator itself dominates the inter-frame change.

6.3 Discussion and limitations

The results support a narrower claim than the framework as a whole: on interfaces with the visual grammar of a Connected TV application, judging semantic structure against an intent-derived oracle separates harmless rendering variation from focus, hierarchy and motion defects far better than comparing pixels to a reference bitmap, and it does so with a deterministic rule whose failures can be attributed to a single term. Several limitations bound the claim. The benchmark is synthetic. Its renderer was written by the authors, the defect classes were injected rather than harvested from production incidents, and the platform styles emulate rather than reproduce real Roku, Android TV, Tizen and WebOS output; the class mix therefore says nothing about field frequency, and a real device will add capture noise, compression and design variety that the renderer does not model. The proposed method was evaluated without its learned components. Element verification by rule works because the renderer’s visual grammar is known; on a real application the CNN and ViT classifiers of Section 4.2 have to supply that knowledge, and their accuracy will bound the semantic match score. Thresholds were tuned once on a validation split drawn from the same renderer, so the reported test numbers are an in-distribution estimate. The comparison set is two image-similarity baselines; the perceptual, parser and vision-language baselines that a real-screenshot study can support were not run. Finally, the per-frame latency was measured on one CPU core for a Python implementation with an exhaustive offset search, so it is a measurement specific to this implementation and hardware configuration.

7 Conclusion

This paper presented a Semantic Visual Validation framework for OTT user interfaces on Connected TV. Frames captured from a device are normalized for HDR and platform rendering and judged through three scored quantities, focus correctness, intent-derived semantic match and temporal smoothness, that combine into the deterministic verdict of Eq. (9). On a synthetic benchmark of 624 held-out instances that emulates four platform focus conventions across home, browse, detail and playback screens, a non-learned instantiation of that verdict reduced the false positive rate on benign variation from 70.5 percent for pixel-diff to 6.8 percent, raised defect recall from 69.8 to 95.5 percent, and reached an F1 of 0.939 against 0.554 for pixel-diff and 0.525 for structural similarity, at 96.8 ms per frame on one CPU core. The gain comes from decoupling the verdict from pixel counts: the baselines can match the recall of the rule only at thresholds that flag most correct frames, while the rule leaves benign variation alone and is the only method that detects most overlap, truncation and gradient defects. Its remaining false alarms on benign transitions and misses on hidden truncation are attributable to named terms of the rule and are fixable in the instantiation. Test oracles derived from the design specification removed bitmap baseline maintenance from the workflow. The limitations are a synthetic benchmark written by the authors, an evaluation without the learned focus and element models, thresholds tuned in distribution, a comparison set limited to two image-similarity baselines, and a latency figure from an unoptimized single-core implementation. Future work will first replace

the benchmark with labeled captures from real devices and train the focus and element models on them, then add the perceptual, parser-based and vision-language baselines that real screenshots permit, and only then pursue the pruned and quantized edge models, the audio-visual feedback checks and the real-time authoring recommendations that the architecture was designed to support.

Acknowledgments

The authors thank colleagues and practitioners in the OTT and Connected TV field for discussions and feedback during the preparation of this work, and the open-source scientific Python, OpenCV and Graphviz communities whose tools were used to build the benchmark, the evaluation and the figures. AI tools supported manuscript drafting and revision, mathematical exposition, and diagram specifications.

References

1. Wang, Z., Bovik, A.C., Sheikh, H.R., Simoncelli, E.P.: Image quality assessment: from error visibility to structural similarity. *IEEE Transactions on Image Processing* 13(4), 600–612 (2004). <https://doi.org/10.1109/TIP.2003.819861>
2. Zhang, R., Isola, P., Efros, A.A., Shechtman, E., Wang, O.: The unreasonable effectiveness of deep features as a perceptual metric. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 586–595 (2018). <https://doi.org/10.1109/CVPR.2018.00068>
3. Liu, Z., Chen, C., Wang, J., Huang, Y., Hu, J., Wang, Q.: Owl Eyes: spotting UI display issues via visual understanding. In: *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 398–409 (2020). <https://doi.org/10.1145/3324884.3416547>
4. Deka, B., Huang, Z., Franzen, C., Hibschan, J., Afegan, D., Li, Y., Nichols, J., Kumar, R.: Rico: a mobile app dataset for building data-driven design applications. In: *Proceedings of the 30th Annual ACM Symposium on User Interface Software and Technology (UIST)*, pp. 845–854 (2017). <https://doi.org/10.1145/3126594.3126651>
5. Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., Houlsby, N.: An image is worth 16×16 words: transformers for image recognition at scale. In: *International Conference on Learning Representations (ICLR)* (2021). <https://openreview.net/forum?id=YicbFdNTTy>
6. Lu, Y., Yang, J., Shen, Y., Awadallah, A.: OmniParser for pure vision based GUI agent. *arXiv preprint arXiv:2408.00203* (2024). <https://arxiv.org/abs/2408.00203>
7. Qin, Y., Ye, Y., Fang, J., Wang, H., Liang, S., Tian, S., Zhang, J., Li, J., Li, Y., Huang, S., et al.: UI-TARS: pioneering automated GUI interaction with native agents. *arXiv preprint arXiv:2501.12326* (2025). <https://arxiv.org/abs/2501.12326>
8. Wang, S., Liu, W., Chen, J., Zhou, Y., Gan, W., Zeng, X., Che, Y., Yu, S., Hao, X., Shao, K., Wang, B., Wu, C., Wang, Y., Tang, R., Hao, J.: GUI agents with foundation models: a comprehensive survey. *arXiv preprint arXiv:2411.04890* (2025). <https://arxiv.org/abs/2411.04890>

9. Zhang, L., Le, B., Zhao, P., Akhtar, N.: Beyond pixel diffs: benchmarking image change captioning for web UI visual regression testing. arXiv preprint arXiv:2607.01728 (2026). <https://arxiv.org/abs/2607.01728>
10. Su, Y., Pradel, M., Chen, C.: RippleGUItester: change-aware exploratory testing. arXiv preprint arXiv:2603.03121 (2026). <https://arxiv.org/abs/2603.03121>
11. Gao, Y., Wu, J., Chen, X., Yang, Y., Cui, Z., Ma, T., Zhang, J., Sang, J.: GUITester: enabling GUI agents for exploratory defect discovery. arXiv preprint arXiv:2601.04500 (2026). <https://arxiv.org/abs/2601.04500>
12. Teed, Z., Deng, J.: RAFT: recurrent all-pairs field transforms for optical flow. In: Vedaldi, A., Bischof, H., Brox, T., Frahm, J.M. (eds.) Computer Vision – ECCV 2020. LNCS, vol. 12347, pp. 402–419. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-58536-5_24
13. Reinhard, E., Stark, M., Shirley, P., Ferwerda, J.: Photographic tone reproduction for digital images. ACM Transactions on Graphics 21(3), 267–276 (2002). <https://doi.org/10.1145/566654.566575>
14. Xu, H., Zhang, X., Liu, H., Wang, J., Zhu, Z., Zhou, S., Hu, X., Gao, F., Cao, J., Wang, Z., et al.: Mobile-Agent-v3.5: multi-platform fundamental GUI agents. arXiv preprint arXiv:2602.16855 (2026). <https://arxiv.org/abs/2602.16855>